arXiv:2610.28811v1 [cs.LG] 20 Oct 2026

A Mechanistic Study of Automated Failure Attribution in LLM-based Agentic Systems

Wendy Zheng[1], Liang Wu[2], Yinhan He[1], Mayank Darbari[2], Jundong Li[1], and Liangjie Hong[2]

[1]University of Virginia, [2]Nokia

Abstract

Agentic systems have demonstrated strong capabilities on complex tasks by enabling multiple interacting large language model (LLM)-based agents to collaborate and execute subtasks in parallel. However, this collaborative structure introduces new challenges for reliably evaluating and diagnosing system failures, particularly failure attribution: tracing a system's execution to identify the agent step responsible for an incorrect final outcome. Because manually performing this task is time- and labor-intensive, recent work has turned to LLMs as judges to automate the process, referred to as automated failure attribution (AFA). These approaches assume that a sufficiently capable model can infer the failing step directly from an interaction transcript. In practice, however, direct prompting often performs poorly, which motivates follow-up work that explores various ways to improve AFA performance, including alternative prompting strategies and inference-time interventions. In this paper, we examine a more fundamental question: whether this weak behavioral performance reflects a genuine capability limitation. Using linear probes applied to internal activations, we uncover a significant *knows-but-won't-say gap:* models encode sufficient information to attribute a failure correctly but fail to explicitly output the responsible step. Building on this finding, we propose *grafting*, a simple manual intervention that exploits knowledge already encoded in the model to improve AFA performance without requiring ground-truth labels at inference time. Extensive evaluations across multiple LLMs and benchmark settings demonstrate that grafting consistently outperforms direct-prompting approaches, underscoring the potential of leveraging latent model knowledge for more reliable failure attribution. Our code is available at `https://github.com/wendyzheng22/mi4afa`.

1 Introduction

Agentic systems extend the capabilities of large language models (LLMs) by decomposing complex tasks and distributing them across multiple specialized agents (e.g., planners, retrievers, coders, and verifiers), that collaborate and execute subtasks in parallel (Acharya et al., 2025; Bandi et al., 2025; Wu et al., 2023). This collaborative paradigm enables agentic systems to tackle problems that would exceed the capacity of a single model working alone. However, as the deployment of such systems expands into high-stakes settings, reliable evaluation methods become essential (Tran et al., 2025; Gridach et al., 2025). One key evaluation task is failure attribution: identifying the earliest step in a system's trajectory that caused the final output to fail. Performing this task manually is expensive, since agentic systems can involve many interacting agents, and the space of possible interactions grows exponentially with system size. This challenge motivates Automated Failure Attribution (AFA) (Zhang et al., 2025), which employs an LLM as a judge to identify the failing step directly from a system's interaction transcript. However, as demonstrated by the Who&When benchmark (Zhang et al., 2025), even large, capable LLMs perform poorly on this task.

This weak performance has spurred a line of follow-up work that seeks to improve AFA, such as refined prompting strategies (West et al., 2025; Sun et al., 2026) or inference-time interventions (Ma et al., 2026). These approaches share a common underlying assumption: the capability for failure attribution already exists within the model and simply needs to be elicited more effectively. However, the persistently poor performance of direct prompting raises a more fundamental question: do LLMs actually possess the capability required for AFA? To answer this, we look beyond the model's generated output and instead examine the information encoded in its internal representations using simple linear probes Alain & Bengio (2016). Surprisingly, these probes recover substantially more information about the correct failing step than the model's generated answer. We refer to this discrepancy as the *knows-but-won't-say gap*: LLMs encode considerably more AFA-relevant information internally than they are able to express behaviorally. This finding indicates that poor AFA performance does not necessarily reflect an absence of underlying capability; rather, the relevant information may already be present within the model but not effectively accessed during generation.

This observation then raises a second question: if models already encode the information needed for AFA, can this latent capability be translated into better behavioral performance? To investigate this question, we propose *grafting*, a simple manual intervention that connects internal components encoding information about the failing step to the components involved in generating the model's output. Rather than teaching the model new failure-attribution knowledge, grafting improves how existing information is routed during generation, enabling the model to more effectively express knowledge it already possesses. In this way, grafting converts latent, decodable AFA information into improved behavioral predictions without requiring ground-truth labels at inference time.

We evaluate direct prompting, linear probing, and grafting across multiple open-weight LLMs and benchmark settings, and find a consistent progression: direct prompting provides a weak behavioral baseline; linear probes reveal that substantially more information about the correct failing step is encoded in model activations; and grafting converts a significant portion of this latent information into improved behavioral predictions. Together, these findings suggest that the limitations of AFA stem not solely from whether models contain the relevant information, but also from whether that information can be effectively accessed. Our contributions are summarized as follows:

- **Knowledge–behavior gap.** We show that linear probes consistently outperform direct prompting across model–dataset combinations, revealing a substantial gap between the AFA-relevant information encoded in model activations and the information expressed through behavioral predictions. This provides evidence that LLMs possess considerably more AFA-relevant capability than their direct responses reflect.
- **Grafting.** We introduce *grafting*, a simple manual intervention that bridges components that encode information about the failing step to those involved in output generation. Grafting consistently improves AFA performance over direct prompting without requiring ground-truth labels at inference time.

2 Background and Preliminaries

In this section, we formally define the Automated Failure Attribution (AFA) task and establish the performance of direct prompting as a baseline. These results motivate the central question of this work: whether the poor behavioral performance of LLMs on AFA reflects a genuine lack of capability, or merely a failure to express information already encoded within the model.

2.1 Automated Failure Attribution

Consider an agentic system composed of one or more interacting agents that is given a task but produces an incorrect final answer. Let $H = (h_1, h_2, \ldots, h_n)$ denote the resulting interaction transcript, where each h_i corresponds to the output of one agent at step i. Each transcript is annotated with the earliest step $e \in \{1, \ldots, n\}$ whose error ultimately causes the system to fail, which referred to as the *mistake step* (equivalently, the *failure step*). Given the transcript H, the goal of AFA is to predict the failing step $\hat{e} \in \{1, \ldots, n\}$. An AFA method succeeds when $\hat{e} = e$, correctly identifying the earliest mistake responsible for the incorrect final output.

Following Zhang et al. (2025), we report two evaluation metrics throughout the paper: (1) **Step accuracy** measures the probability of correctly identifying the failing step, $P(\hat{e} = e)$, and (2) **Agent accuracy** measures whether the agent at the predicted mistake step $\hat{e}$ matches the ground-truth failing agent, regardless of which occurrence of that agent is selected. Because a single agent may speak multiple times over the course of a transcript, a prediction can identify the wrong step while still attributing failure to the correct agent. Agent accuracy is therefore a strictly coarser metric than step accuracy, and the two do not necessarily move together.

2.2 Direct Prompting

We first evaluate direct prompting to characterize the behavioral baseline of LLMs on the AFA task, using the Who&When benchmark (Zhang et al., 2025). This benchmark consists of two subsets: the **Algorithm-generated (Algo.)** subset, containing relatively short execution traces produced automatically, and the **Hand-crafted (Craft.)** subset, containing substantially longer and more complex agent trajectories designed by human annotators.

Table 1 summarizes the performance of several open-weight and closed-source LLMs under direct prompting. The closed-source LLMs are accessed via Cursor Python SDK (Anysphere, 2026). Two observations are notable. First, consistent with prior findings, every model achieves less desirable step accuracy on both subsets, confirming that reliably identifying the failing step remains difficult regardless of model family. Second, performance does not scale consistently with model size or capability: within the Qwen family, the 1.5B model underperforms the 0.5B variant on step accuracy for Algo., and the closed-source models only modestly outperform substantially smaller open-weight models on Craft. This absence of a clear capacity trend suggests that increasing model size alone is insufficient to close the gap in behavioral performance on AFA, motivating the investigation, in the following sections, of what the model encodes internally rather than what direct prompting can elicit from it.

3 Method

In this section, we introduce two methods to address the questions posed in Section 1: (1) *probing*, which determines whether LLMs internally encode sufficient information to identify the failing step, and (2) *grafting*, which investigates whether this latent information can be exposed to improve the model's behavioral performance.

3.1 Probing: Is the failing step decodable from model activations?

Poor performance under direct prompting does not necessarily mean that an LLM lacks the capability required for AFA. The relevant information may instead be encoded in its hidden representations without being reflected in its output distribution. To distinguish between these possibilities, we train

Table 1: Direct Prompting Results on the Who&When benchmark

	Algorithm Generated		Hand Crafted	
	Agent Acc.	Step Acc.	Agent Acc.	Step Acc.
Qwen 0.5B	41.27	22.22	39.29	7.14
Qwen 1.5B	33.33	7.94	46.43	10.71
Qwen 7B	30.16	12.70	46.43	0.00
Llama 1B	23.81	15.87	17.86	3.57
Llama 3B	30.16	17.46	25.00	7.14
Gemma 1B	28.57	9.52	32.14	3.57
Gemma 4B	26.98	7.94	60.71	7.14
GPT-5.6 Sol	61.90	50.79	57.14	25.00
GPT-5.6 Terra	65.08	52.38	67.86	35.71
Sonnet 5	57.14	42.86	67.86	28.57
Opus 4.8	61.90	46.03	78.57	39.29
Opus 5	66.67	50.79	82.14	35.71

linear probes to test whether the failing step can be linearly decoded from intermediate activations. Our probing pipeline has three stages: activation extraction, probe training, and prediction.

Activation extraction. For each transcript, we perform a single forward pass through the frozen LLM and extract hidden representations from every transformer layer at two sites: the post-attention residual stream and the post-MLP residual stream. For a model with L layers, this gives $2L$ candidate representation sites. Because AFA is a step-level prediction problem, each step is represented by the hidden state x at the final token of that step.

Probe training. We evaluate each of the $2L$ representation sites independently. For a fixed site, we pool the hidden states x from every step in every training transcript into a single dataset. Each hidden state receives a binary label indicating whether its corresponding step is the ground-truth failing step:

$$y = \begin{cases} 1, & \text{if the step is the ground-truth failing step,} \\ 0, & \text{otherwise.} \end{cases}$$

We train a linear logistic probe

$$s(x) = w^\top \hat{x} + b, \tag{1}$$

where

$$\hat{x} = \frac{x - \mu}{\sigma}, \tag{2}$$

and μ and σ are computed only on the training split. The probe minimizes

$$\mathcal{L}(w, b) = \mathbb{E}\left[\mathrm{BCE}(s(\hat{x}), y; \pi)\right] + \lambda \|w\|_2^2,$$

where

$$\pi = \frac{N - N_+}{N_+}$$

is the positive-class weight used to compensate for the severe class imbalance created by each transcript having exactly one labeled failure step.

Prediction. At evaluation, the trained probe independently scores every step in a transcript using the corresponding hidden representation. The predicted failing step is the highest-scoring step: $\hat{e} = \arg\max_k s(x_k)$. Evaluating probes at every residual location allows us to identify where failure-attribution information is most linearly accessible within the model.

3.2 Grafting: Making the model say what it locates

Linear probing reveals what information is encoded in the model but leaves its behavior unchanged. The goal now is to improve the model's own predictions. Our approach is motivated by a simple hypothesis about the functional organization of transformer attention heads: some heads specialize in locating the failing step (locator heads), while others specialize in writing the model's predicted answer into the output distribution (writer heads). If these two groups are only weakly coupled, the model may internally identify the correct failing step without effectively communicating this information to the output. Grafting aims to bridge this gap using a three step process: (1) identify locator heads, (2) identify writer heads, and (3) connect them through an inference-time intervention.

Step 1: Identify locator heads We define locator heads as attention heads whose attention patterns reliably identify the failing step. Intuitively, if a head consistently places most of its attention on the ground-truth mistake step when producing the final answer, then that head provides a useful signal about where the failure occurred. Thus, for each head (ℓ, h), we compute the attention distribution at the last token position:

$$z^{(\ell,h)} = \mathrm{softmax}\left(\frac{q_{\mathrm{last}}^{(\ell,h)} K^{(\ell,h)}}{\sqrt{d_h}}\right), \tag{3}$$

where d_h represents the head dimension. Summing the attention mass over each step's token span gives the total weight assigned to step s:

$$\alpha_s^{(\ell,h)} = \sum_{t=\tau_t}^{\tau_{t+1}-1} z_t^{(l,h)}, \qquad s = 1, \ldots, n, \tag{4}$$

where $\tau_s$indexes the token boundaries of step s. Each head casts a vote for the step receiving the largest attention mass,

$$\arg\max_s \alpha_s^{(\ell,h)}.$$

A head's *locator score* is the proportion of training examples on which its vote matches the ground-truth failing step. We rank heads by this score and retain at most five heads from the top 10% of the ranking, excluding heads in the final transformer layer (a final-layer head has no downstream computation left to route its signal into, so it cannot itself act as a useful locator for the graft).

Step 2: Identify writer heads Whereas locator heads determine *which* step is important, writer heads determine *what* the model ultimately outputs. We therefore identify these using direct logit attribution (nostalgebraist, 2020), which measures a component's direct contribution to a particular output logit. Each attention and MLP layer's output is added into the residual stream, so the final residual stream $\mathrm{resid}^{(L)}$ is a linear sum of every component's contribution. Projecting a component's

output through the model's final normalization scale g and the unembedding matrix W_U gives that component's direct, linear contribution to any vocabulary logit v:

$$\mathrm{DLA}^{(\ell,h)}(v) \;=\; \frac{\big(W_O^{(\ell,h)} z_{\text{last}}^{(\ell,h)}\big)^\top r(v,C)}{\mathrm{rms}\big(\mathrm{resid}_{\text{last}}^{(L)}\big)}. \tag{5}$$

where $z_{\text{last}}^{(\ell,h)}$ is the output of an attention head at the last token (before the output projection W_O), and $r(v,C)$ is a contrastive readout direction defined as

$$r(v,C) = g \odot \Big(W_U[v] - \tfrac{1}{|C|}\textstyle\sum_{c\in C} W_U[c]\Big), \tag{6}$$

where C is the set of candidate step-index tokens for the transcript. Subtracting the mean unembedding vector over the candidate tokens removes directions shared across candidates and emphasizes the dimensions that distinguish the target token from the other possible step indices. Writer heads are scored by their DLA toward the step index token that the model emits on the clean forward pass, instead of the ground truth mistake step. We intentionally measure contribution toward the model's own predicted answer because the objective is to identify the components currently responsible for expressing the model's internal belief, irrespective of whether that belief is correct. Writer heads are identified on the training set, keeping up to five heads from the top 10%, excluding heads in the first layer (a first-layer head has not yet had access to enough contextual computation to serve as a meaningful writer).

Step 3: Construct the graft Once the locator and writer heads have been identified, grafting combines them at inference time without any supervision or ground-truth labels:

1. **Read the locators.** Each selected locator head votes for its top-attended step; we take the plurality winner $\hat{t}$ as the model's internal guess. We use a vote across heads rather than relying on a single head because individual heads are not perfectly reliable.

2. **Form the target direction.** Map $\hat{t}$ to its index token and compute $r(\hat{t})$ by Equation 6.

3. **Pull it back into head space.** For each writer head i, we compute the direction in its value space that maximally increases the target logit after the head's output projection: $v_i = W_O^{(i)\top} r(\hat{t})$.

4. **Inject.** At the final token position, add v_i to the writer's value vector before W_O, scaled relative to the norm of the full concatenated head output z_{last} at that position:

$$z_i \;\leftarrow\; z_i \;+\; \beta\,\frac{\|z_{\text{last}}\|}{\|v_i\|}\,v_i. \tag{7}$$

   Here, β controls the strength of the intervention, while the norm ratio scales the injected direction relative to the magnitude of the model's existing head output.

The intervention is applied in the writer heads' value space rather than directly to the residual stream. As a result, the injected signal passes through the same output projection W_O that the model normally uses to write information from the head into the residual stream. Together, these steps create a bridge between the model's internal localization and its output behavior.

Table 2: Performance comparison between different prompting strategies, probing, and grafting

			Prompting			Probing	Grafting
			All-at-Once	Step-by-step	Sequential		
Algo.	Qwen 0.5B	Agent Acc.	34.92	41.27	47.62	53.97	**61.90**
		Step Acc.	23.81	22.22	14.29	36.51	**38.10**
	Qwen 1.5B	Agent Acc.	30.16	33.33	50.79	**57.14**	**57.14**
		Step Acc.	20.63	7.94	6.35	**44.44**	33.33
	Qwen 7B	Agent Acc.	28.57	30.16	39.68	**61.90**	49.21
		Step Acc.	11.11	12.70	14.29	**47.62**	30.16
	Llama 1B	Agent Acc.	25.40	23.81	30.16	53.97	**60.32**
		Step Acc.	20.63	15.87	17.46	**36.51**	34.92
	Llama 3B	Agent Acc.	38.10	30.16	50.79	**60.32**	53.97
		Step Acc.	22.22	17.46	12.70	**46.03**	34.92
	Gemma 1B	Agent Acc.	28.57	28.57	23.81	**63.49**	39.68
		Step Acc.	14.29	9.52	9.52	**42.86**	23.81
	Gemma 4B	Agent Acc.	30.16	26.98	60.32	**61.90**	41.27
		Step Acc.	11.11	7.94	11.11	**44.44**	22.22
Craft.	Qwen 0.5B	Agent Acc.	39.29	39.29	28.57	**67.86**	64.29
		Step Acc.	10.71	7.14	7.14	25.00	**28.57**
	Qwen 1.5B	Agent Acc.	32.14	46.43	39.29	**75.00**	57.14
		Step Acc.	3.57	10.71	17.86	**25.00**	**25.00**
	Qwen 7B	Agent Acc.	28.57	46.43	**64.29**	60.71	32.14
		Step Acc.	0.00	0.00	7.14	**28.57**	10.71
	Llama 1B	Agent Acc.	21.43	17.86	14.29	**57.14**	35.71
		Step Acc.	3.57	3.57	0.00	**21.43**	**21.43**
	Llama 3B	Agent Acc.	17.86	25.00	57.14	**75.00**	46.43
		Step Acc.	0.00	7.14	7.14	**28.57**	25.00
	Gemma 1B	Agent Acc.	32.14	32.14	7.14	**60.71**	57.14
		Step Acc.	3.57	3.57	3.57	17.86	**21.43**
	Gemma 4B	Agent Acc.	42.86	60.71	50.00	**75.00**	67.86
		Step Acc.	3.57	7.14	3.57	**21.43**	17.86

4 Experiments

In this section, we first describe our experimental setup and then present results comparing the performance of probing and grafting to the behavioral prompting baselines.

4.1 Experiment Setup

We evaluate seven open-weight LLMs spanning three model families and multiple model sizes: Qwen 2.5 0.5B, 1.5B, and 7B Instruct variants (Yang et al., 2024; Team, 2024; Qwen, 2024a,b,c); Llama 3.2 1B and 3B variants (Meta, 2024a,b); and Gemma 3 1B and 4B variants (Deepmind, 2025a,b; Team et al., 2025). This selection allows us to examine AFA performance across both model scale and model family.

Evaluation Tasks. We adopt the Who&When benchmark (Zhang et al., 2025), which contains two subsets with different trajectory lengths. The Algorithm Generation (Algo.) subset contains transcripts with an average of 9 steps, while the Hand Crafted (Craft.) subset contains substantially longer transcripts with an average of 29 steps. This variation allows us to evaluate AFA across agentic trajectories of different complexity.

Baselines. We compare against three prompting-based baselines. All-at-Once provides the entire transcript to the model at once, and Step-by-Step incrementally presents each step to the model to determine whether it constitutes the failure; both strategies were introduced by Zhang et al. (2025). Sequential decomposes attribution into two stages, first identifying the responsible agent and subsequently predicting the failing step conditioned on that agent.

Computation Resources. All experiments were performed on a cluster of four A100 80GB PCIe GPUs running Cuda version 13.0.

4.2 Failure-Attribution Information is Linearly Decodable

Table 2 compares the performance of all five methods. We first examine whether the poor performance of direct prompting reflects a genuine absence of failing-step information within the model. Across the evaluated models, linear probes substantially outperform the prompting baselines on step accuracy. For Qwen 0.5B on Algo., for example, the strongest prompting baseline reaches 23.81% step accuracy, compared with 36.51% for probing. For Qwen 1.5B and Qwen 7B, probing reaches 44.44% and 47.62%, respectively, compared with 15.87% and 14.29% for the strongest prompting baseline. Comparable gains are observed for Llama and Gemma.

These results indicate that failing-step information is substantially more accessible from the model's internal representations than from its generated answers. A simple linear classifier can recover information about the correct step that the model's own output does not reliably express. We refer to this discrepancy between internally encoded latent information and behavioral performance as the *knows-but-won't-say* gap.

At the same time, the results reveal substantial variation in the behavioral baselines across the different configurations. No prompting strategy consistently performs best, and increasing model size does not uniformly translate into better failure attribution. This variability further suggests that the primary challenge may not simply be acquiring failure-attribution information, but effectively accessing and expressing information that is already encoded in the model.

4.3 Grafting Exploits Latent Information for Improved Performance

The probing results establish that LLMs encode information about the failing step, but probing itself does not provide a mechanism for using this information during generation. This motivates

our next approach, grafting, which aims to leverage the model's own output mechanism to express its predicted failing step. As shown in Table 2, grafting consistently outperforms the prompting baselines and substantially narrows the gap between behavioral prediction and the information revealed by probing. For example, on the Algo. subset, grafting achieves the highest step accuracy for Qwen 0.5B (38.10%), while also substantially outperforming all prompting methods for Qwen 1.5B (33.33% vs. 15.87% for Step-by-Step) and Qwen 7B (30.16% vs. 14.29% for Sequential). Similar improvements are observed for the Llama and Gemma models. On the Craft. subset, grafting also improves step accuracy over prompting for several models, including Qwen 0.5B (28.57%), Gemma 1B (21.43%), and Llama 3B (21.43%).

Importantly, grafting does not simply reproduce the performance of the linear probe: probing remains stronger in many settings. Rather, the results show that a substantial portion of the information that is linearly decodable from the model can be converted into improved behavioral predictions through a targeted intervention on the model's internal computation. This provides evidence that the knowledge–behavior gap is not merely an observation about representational capacity, but can be actively exploited to improve AFA.

5 Related Works

Automated Failure Attribution Automated Failure Attribution (AFA) aims to identify the agent and step responsible for a task failure in an LLM-based multi-agent system (Zhang et al., 2025). The Who&When benchmark demonstrates a substantial gap between agent-level and step-level attribution(Zhang et al., 2025). Thus, subsequent work aims to improve AFA performance through diverse strategies, including decomposing attribution into scope delineation and localization (Sun et al., 2026), causal counterfactual reasoning (West et al., 2025), and intervention-based verification of failure hypotheses (Ma et al., 2026). Other recent approaches further investigate automated localization and hierarchical attribution (Zhang et al., 2026; Banerjee et al., 2025). More recently, TraceElephant evaluates attribution under full execution observability, showing that access to complete execution traces can substantially improve attribution accuracy (Chen et al., 2026). These approaches primarily seek to improve an evaluator's ability to produce correct attribution judgments from trajectory information. We instead ask whether AFA-relevant information is already encoded in the evaluator's internal representations and whether it can be routed into generation.

Latent Knowledge and Probing Prior work has shown that language models can encode information that is not reliably expressed in their outputs. Probing classifiers provide a standard method for testing whether properties are linearly recoverable from intermediate representations (Alain & Bengio, 2016; Belinkov, 2022). In particular, latent knowledge can be recovered from activations even when models are prompted to produce incorrect answers (Burns et al., 2022), and hidden states encode signals related to truthfulness (Azaria & Mitchell, 2023) with approximately linear structure (Marks & Tegmark, 2023). These results motivate the distinction between information that is decodable from representations and information that is actually expressed or used during generation. We apply this distinction to AFA, using linear probes to quantify attribution information encoded in intermediate representations and testing whether this information can be made behaviorally accessible during generation.

Activation Intervention and Representation Engineering A complementary literature studies interventions on internal representations to with the goal of altering model behavior. Activation engineering and representation engineering manipulate the model's intermediate representations

to steer its outputs (Turner et al., 2023; Zou et al., 2023), while inference-time intervention uses directions identified from activation-space classifiers to steer models toward more truthful outputs (Li et al., 2023). Activation patching and causal tracing provide complementary methods for localizing model components involved in specific computations through causal interventions (Meng et al., 2022; Zhang & Nanda, 2024). Grafting builds on these ideas but targets a different problem: rather than steering toward a predefined behavioral attribute, it connects representations from which a task-specific variable (i.e., the failing step)is decodable to components involved in output generation. Thus, grafting tests whether information already decodable from a model's representations can be translated into improved task behavior.

6 Conclusion

In this paper, we study Automated Failure Attribution (AFA) from the perspective of the gap between information encoded in LLM representations and information expressed through their behavior. We show that linear probes consistently recover substantially more information about the failing step than direct prompting, revealing a *knows-but-won't-say gap* in LLM-based failure attribution. Motivated by this observation, we propose *grafting*, a simple inference-time intervention that connects components encoding failing-step information to those responsible for output generation, allowing the model to more effectively express its internally encoded knowledge. Experiments across multiple LLMs and benchmark settings demonstrate that grafting improves behavioral AFA performance over direct prompting, showing that latent failure-attribution information can be translated into improved predictions without requiring ground-truth labels at inference time. More broadly, these findings suggest that improving LLM-based evaluators may depend not only on scaling models or refining prompts, but on developing methods that allow models to more effectively access and express knowledge they already possess.

References

Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. *IEEe Access*, 13:18912–18936, 2025.

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

Inc. Anysphere. cursor-sdk: The official python sdk for cursor agent workflows, 2026. URL `https://pypi.org/project/cursor-sdk/`.

Amos Azaria and Tom Mitchell. The internal state of an llm knows when it's lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 967–976, 2023.

Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet*, 17(9):404, 2025.

Adi Banerjee, Anirudh Nair, and Tarik Borogovac. Where did it all go wrong? a hierarchical look into multi-agent error attribution. *arXiv preprint arXiv:2510.04886*, 2025.

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.

Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022.

Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the whole elephant: A benchmark for failure attribution in llm-based multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 19888–19905, 2026.

Google Deepmind. gemma-3-1b-it, 2025a. URL `https://huggingface.co/google/gemma-3-1b-it`.

Google Deepmind. gemma-3-4b-it, 2025b. URL `https://huggingface.co/google/gemma-3-4b-it`.

Mourad Gridach, Jay Nanavati, Khaldoun Zine El Abidine, Lenon Mendes, and Christina Mack. Agentic ai for scientific discovery: A survey of progress, challenges, and future directions. *arXiv preprint arXiv:2503.08979*, 2025.

Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. *Advances in neural information processing systems*, 36:41451–41530, 2023.

Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Dover: Intervention-driven auto debugging for llm multi-agent systems. In *International Conference on Learning Representations*, volume 2026, pp. 28059–28087, 2026.

Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. *arXiv preprint arXiv:2310.06824*, 2023.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022.

Meta. Llama-3.2-1b, 2024a. URL `https://huggingface.co/meta-llama/Llama-3.2-1B`.

Meta. Llama-3.2-3b, 2024b. URL `https://huggingface.co/meta-llama/Llama-3.2-3B`.

nostalgebraist. interpreting gpt: the logit lens, 2020. URL `https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens`.

Qwen. Qwen2.5-0.5b-instruct, 2024a. URL `https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct`.

Qwen. Qwen2.5-1.5b-instruct, 2024b. URL `https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct`.

Qwen. Qwen2.5-7b-instruct, 2024c. URL `https://huggingface.co/Qwen/Qwen2.5-7B-Instruct`.

Kai Sun, Wenqiang Li, Bo Dong, Yuxin Lin, Jingyao Zhang, and Bin Shi. Scope delineation before localization: A two-stage framework for enhancing failure attribution in multi-agent systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 33108–33116, 2026.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL `https://qwenlm.github.io/blog/qwen2.5/`.

Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D Nguyen. Multi-agent collaboration mechanisms: A survey of llms. *arXiv preprint arXiv:2501.06322*, 2025.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023.

Alva West, Yixuan Weng, Minjun Zhu, Zhen Lin, Zhiyuan Ning, and Yue Zhang. Abduct, act, predict: Scaffolding causal inference for automated failure attribution in multi-agent systems. *arXiv preprint arXiv:2509.10401*, 2025.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*, volume 2024, pp. 1651–1678, 2024.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. Agentracer: Who is inducing failure in the llm agentic systems? In *International Conference on Learning Representations*, volume 2026, pp. 11377–11399, 2026.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, et al. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. *arXiv preprint arXiv:2505.00212*, 2025.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.